

Node.js Async

Promises, Event Emitters, Concurrency & Streams

Lessons with runnable examples

1 • Promises

A **Promise** represents a value that may not be available yet. It is always in one of three states:

- **pending** — not yet settled
- **fulfilled** — completed, has a value (`resolve` was called)
- **rejected** — failed, has a reason (`reject` was called)

Once a promise **settles** it never changes state again. That immutability is the core mental model.

Creating a promise

The executor function runs **immediately** (synchronously). You call `resolve(value)` on success or `reject(error)` on failure.

```
const coinFlip = new Promise<string>((resolve, reject) => {
  const landed = Math.random() > 0.5 ? "heads" : "tails";
  if (landed === "heads") resolve(landed);
  else reject(new Error("got tails"));
});

// Consuming it: .then on fulfill, .catch on reject, .finally always.
await coinFlip
  .then((r) => "fulfilled: " + r)
  .catch((e) => "rejected: " + e.message);
```

Promises are asynchronous

Even an **already-resolved** promise defers its `.then` callback to the microtask queue. Predict the output order before running:

```
const out: string[] = [];
out.push("A - sync, first");
Promise.resolve("done").then(() => out.push("C - microtask, last"));
out.push("B - sync, second");
// Give the microtask a tick to run, then show the order:
await Promise.resolve();
out; // expect: A, B, C
```

Promisifying a timer

This is how you wrap old callback/timer APIs into promises. `delay` is reused throughout this notebook.

```
function delay<T>(ms: number, value?: T): Promise<T> {
  return new Promise((resolve) => setTimeout(() => resolve(value as T), ms));
}
await delay(100, "resolved after 100ms");
```

Chaining & error propagation

Each `.then()` returns a **new** promise. Returning a value passes it down; returning a *promise* pauses the chain until it settles. A `throw` anywhere skips to the nearest `.catch`, then the chain **recovers**.

```
await Promise.resolve(2)
  .then((n) => n * 10) // 20
  .then((n) => n + 5) // 25
  .then((n) => { throw new Error("boom at " + n); })
  .then(() => "SKIPPED") // jumped over
  .catch((e) => "caught: " + e.message)
  .then((msg) => msg + " → chain recovered");
```

The “forgot to return” bug

Without `return`, the chain does **not** wait for the inner promise. Watch the timestamps:

```
const slow = (ms: number) => new Promise((r) => setTimeout(r, ms));
const t0 = performance.now();
await Promise.resolve()
  .then(() => { slow(100); /* BUG: no return → chain does NOT wait */ })
  .then(() => "elapsed ~" + Math.round(performance.now() - t0) + "ms (NOT ~100)");
```

async / await

Sugar over promises. An `async` function **always** returns a promise; `await` pauses until a promise settles. Rejections become thrown errors you catch with `try/catch`.

```
function delay2<T>(ms: number, v?: T) {
  return new Promise<T>((r) => setTimeout(() => r(v as T), ms));
}

async function run() {
  try {
    await delay2(30, "ok");
    throw new Error("something broke");
  } catch (e) {
    return "caught: " + (e as Error).message;
  } finally {
    console.log("finally always runs");
  }
}

await run();
```

The sequential-vs-parallel trap

The #1 `async/await` performance mistake. Independent work should not be `await` ed one-by-one.

```
function delay3<T>(ms: number, v: T) {
  return new Promise<T>((r) => setTimeout(() => r(v), ms));
}

// SLOW: each await waits for the previous (~300ms)
const s0 = performance.now();
await delay3(100, "a"); await delay3(100, "b"); await delay3(100, "c");
const seq = Math.round(performance.now() - s0);

// FAST: all start at once (~100ms)
const p0 = performance.now();
await Promise.all([delay3(100, "a"), delay3(100, "b"), delay3(100, "c")]);
const par = Math.round(performance.now() - p0);

`sequential ~${seq}ms vs parallel ~${par}ms`;
```

Combinators — `all` / `allSettled` / `race` / `any`

These combine multiple promises. Picking the right one is half of writing good concurrent code.

```

function delay4<T>(ms: number, v: T) { return new Promise<T>((r) => setTimeout(() => r(v), ms)); }
function fail(ms: number, m: string) { return new Promise<never>((_, rej) => setTimeout(() => rej(new Error(m)), ms)); }

// all: every result IN ORDER, rejects fast if ANY rejects
const all = await Promise.all([delay4(50, "first"), delay4(10, "third")]);

// allSettled: never rejects, gives every outcome
const settled = await Promise.allSettled([delay4(10, "A"), fail(10, "B exploded")]);

// race: first to settle (success OR failure) – basis of timeouts
const race = await Promise.race([delay4(100, "slow"), delay4(20, "fast")]);

// any: first to SUCCEED; AggregateError only if all fail
const any = await Promise.any([fail(10, "down"), delay4(40, "mirror2 ok")]);

({ all, settled, race, any });

```

Pitfalls cheat-sheet

Bug	Fix
<code>await</code> inside a <code>for</code> loop for independent work	<code>Promise.all(items.map(fn))</code>
<code>array.forEach(async ...)</code> — does not await	<code>for...of</code> + <code>await</code> , or <code>map</code> + <code>Promise.all</code>
Rejected promise with no <code>.catch</code>	always handle — Node/Deno crash on unhandled rejection
<code>throw "string"</code>	<code>throw new Error(...)</code> — strings have no stack
<code>new Promise((res) => fetch().then(res))</code>	just <code>return fetch()</code> — don't wrap a promise

You've finished Promises. Next: **2 • Event Emitters**.

2 • Event Emitters

The **publish/subscribe** (observer) pattern. Objects **emit** named events; functions **subscribe** and run when the event fires. Streams, HTTP servers, sockets — all EventEmitters underneath.

We import Node's `events` module — Deno supports Node built-ins via the `node:` prefix.

```
import EventEmitter from "node:events";

const bus = new EventEmitter();
const log: string[] = [];

// Many listeners per event run in registration order.
bus.on("greet", (name: string) => log.push("A: hello " + name));
bus.on("greet", (name: string) => log.push("B: hi " + name));

bus.emit("greet", "Avindu"); // SYNCHRONOUS — blocks until listeners run
log.push("emit returned (listeners already ran)");
log;
```

`once`, multiple args, removal, introspection

```
import EventEmitter2 from "node:events";
const e = new EventEmitter2();
const log: string[] = [];

e.once("boot", () => log.push("booted (fires once)"));
e.emit("boot"); e.emit("boot"); // 2nd ignored

e.on("order", (id: number, qty: number, item: string) =>
  log.push(`order ${id}: ${qty}x ${item}`));
e.emit("order", 1001, 3, "coffee");

const tick = () => log.push("tick");
e.on("tick", tick); e.emit("tick");
e.off("tick", tick); e.emit("tick"); // removed → no second tick

log.push("events: " + e.eventNames().join(", "));
log;
```

Building your own emitting class

The idiomatic Node pattern: **extend EventEmitter** so your objects emit domain events. The class doesn't know who's listening — that decoupling is the whole point.

```

import EventEmitter3 from "node:events";

class JobQueue extends EventEmitter3 {
  private jobs: { id: number; willFail?: boolean }[] = [];
  private processing = false;

  add(job: { id: number; willFail?: boolean }) {
    this.jobs.push(job);
    this.emit("added", job);
    if (!this.processing) this.run();
    return this;
  }

  private async run() {
    this.processing = true;
    while (this.jobs.length) {
      const job = this.jobs.shift()!;
      try {
        await new Promise((res, rej) =>
          setTimeout(() => job.willFail ? rej(new Error("job " + job.id + " failed")) :
            res(0), 20));
        this.emit("success", job);
      } catch (err) {
        this.emit("error", job, err); // domain error (job + err), not the special contract
      }
    }
    this.processing = false;
    this.emit("drain");
  }
}

const trace: string[] = [];
const q = new JobQueue()
  .on("added", (j) => trace.push("+ queued " + j.id))
  .on("success", (j) => trace.push("✓ " + j.id))
  .on("error", (j, e) => trace.push("x " + (e as Error).message))
  .on("drain", () => trace.push("drained"));

q.add({ id: 1 }); q.add({ id: 2, willFail: true }); q.add({ id: 3 });
await new Promise((r) => setTimeout(r, 120));
trace;

```

The special `error` event

If an emitter emits `"error"` with **no** `error` listener, it **throws** and crashes the process. Always attach one.

```
import EventEmitter4 from "node:events";
const e = new EventEmitter4();
let handled = "";
e.on("error", (err: Error) => { handled = "handled: " + err.message; });
e.emit("error", new Error("boom")); // safe because we have a listener
handled;
```

Bridging events into async/await

`events.once` turns the next emission into an awaitable promise. `events.on` turns a stream of events into an **async iterator** you can `for await` over.

```
import { on, once } from "node:events";
import EventEmitter5 from "node:events";

// once → await a single event
const e1 = new EventEmitter5();
setTimeout(() => e1.emit("ready", "config-loaded"), 30);
const [payload] = await once(e1, "ready");

// on → consume a stream of events
const e2 = new EventEmitter5();
let n = 0;
const timer = setInterval(() => e2.emit("pulse", ++n), 15);
const pulses: number[] = [];
for await (const [v] of on(e2, "pulse")) {
  pulses.push(v);
  if (v >= 3) { clearInterval(timer); break; }
}

({ payload, pulses });
```

Does the loop register a listener per iteration?

No. `on(emitter, name)` is called **once**, before the loop runs, and returns one async iterator. The `for await` loop just pulls values from that single iterator, so exactly **one** listener is attached (regardless of how many times the loop body runs).

Desugared, the loop is:

```
const it = on(e, "pulse"); // called ONCE, attaches one listener
while (true) {
  const { value, done } = await it.next(); // each iteration pulls
  if (done) break;
  // ...loop body...
}
// it.return() runs on break/throw to detach the listener
```

This is **pull-based with buffering**, unlike `e.on("pulse", fn)` which is **push-based**. Internally `on()` registers one listener that pushes each event into a queue; `it.next()` drains that queue (or waits if it is empty). Two consequences:

- Events that fire **while your loop body is busy** are queued, not dropped. A plain `.on()` handler has no such buffer.
- That queue is **unbounded**, so a producer that outpaces the consumer grows memory. Watch for this in real code.

```
import { on } from "node:events";
import EventEmitterBuf from "node:events";

// Prove (1) only ONE listener is attached, and (2) events that arrive
// while the loop body is awaiting are BUFFERED, not lost.
const e = new EventEmitterBuf();
const iter = on(e, "pulse");
const listeners = e.listenerCount("pulse"); // 1, not growing per iteration

// Fire 5 pulses quickly, every 10ms.
let n = 0;
const timer = setInterval(() => e.emit("pulse", ++n), 10);

const slowDelay = (ms: number) => new Promise((r) => setTimeout(r, ms));
const seen: number[] = [];
for await (const [v] of iter) {
  await slowDelay(40); // body is SLOWER than the producer (10ms)
  seen.push(v);        // yet we still see every value, in order
  if (v >= 5) { clearInterval(timer); break; }
}

// listeners === 1 even though the loop ran 5 times; seen is 1..5 with
// none dropped despite the consumer being 4x slower than the producer.
({ listeners, seen });
```

Takeaways: `emit` is synchronous and ignores async-listener completion. Always handle `"error"`. Use `once / on` to cross into the promise world. `on()` attaches a single buffering listener (not one per loop turn), so slow consumers queue rather than drop events, but watch that unbounded queue. Watch the `MaxListeners` warning too; it usually means a leak.

Next: **3 • Concurrency.**

3 • Concurrency

Your JS runs on a **single thread**. Concurrency comes from the **event loop**: many async ops *in flight* at once. Two queues drive ordering:

- **microtasks** — promise callbacks, `queueMicrotask`
- **macrotasks** — `setTimeout`, `setImmediate`, I/O

After each macrotask the entire microtask queue drains before the next macrotask.

```
const order: string[] = [];  
order.push("1 sync start");  
setTimeout(() => order.push("4 setTimeout (macro)"), 0);  
queueMicrotask(() => order.push("3 microtask"));  
order.push("2 sync end");  
// Let both queues flush:  
await new Promise((r) => setTimeout(r, 5));  
order; // 1, 2, 3, 4
```

Blocking the loop is fatal

A long synchronous loop freezes **everything** — no timers, no I/O, no promise resolution.

```
const events: string[] = [];  
setTimeout(() => events.push("timer (was starved during the block)"), 10);  
const start = Date.now();  
while (Date.now() - start < 50) { /* busy-wait 50ms, blocks the thread */ }  
events.push("blocked ~50ms; the 10ms timer could NOT fire until now");  
await new Promise((r) => setTimeout(r, 0));  
events;
```

Orchestration patterns

`delay` simulates async work (network/disk). Compare sequential vs parallel.

```
function delay<T>(ms: number, v: T) { return new Promise<T>((r) => setTimeout(() => r(v), ms)); }

// sequential – only when each step depends on the last (~150ms)
const s0 = performance.now();
await delay(50, 0); await delay(50, 0); await delay(50, 0);
const seq = Math.round(performance.now() - s0);

// parallel – independent work (~50ms)
const p0 = performance.now();
await Promise.all([delay(50, 0), delay(50, 0), delay(50, 0)]);
const par = Math.round(performance.now() - p0);

`sequential ~${seq}ms vs parallel ~${par}ms`;
```

Bounded concurrency (a pool)

Firing 1000 requests at once exhausts sockets/memory. Run at most **N** at a time. This is the everyday batch-job pattern. Results stay in input order.

```
function delay<T>(ms: number, v: T) { return new Promise<T>((r) => setTimeout(() => r(v), ms)); }

async function mapLimit<T, R>(items: T[], limit: number, fn: (x: T, i: number) => Promise<R>) {
  const results = new Array<R>(items.length);
  let cursor = 0;
  let maxActive = 0, active = 0;
  async function worker() {
    while (cursor < items.length) {
      const i = cursor++; // claim next index
      active++; maxActive = Math.max(maxActive, active);
      results[i] = await fn(items[i], i);
      active--;
    }
  }
  await Promise.all(Array.from({ length: Math.min(limit, items.length) }, worker));
  return { results, maxActive };
}

await mapLimit([1, 2, 3, 4, 5], 2, (n) => delay(30, n * 10));
// maxActive should be 2
```

True parallelism — Web Workers / worker threads

Promises give concurrent **I/O** on one thread; they do **not** help CPU-bound work, which blocks the thread. To see why this matters, run the SAME heavy compute two ways and compare.

Version 1: inline (blocks the main thread)

We run the heavy loop three times directly on the main thread. To make the blocking *visible*, we also start a timer that ticks every 10ms. While the compute runs, the event loop is stuck, so the timer **cannot fire** — its ticks are starved until the compute finishes.

```
function heavy(n: number) {
  let sum = 0;
  for (let i = 0; i < n; i++) sum += Math.sqrt(i) * Math.sin(i);
  return sum;
}

// A probe: this SHOULD tick every 10ms. Count how many times it does.
let ticks = 0;
const probe = setInterval(() => ticks++, 10);

const t0 = performance.now();
const results = [heavy(2e7), heavy(2e7), heavy(2e7)]; // all on the main thread
const elapsed = Math.round(performance.now() - t0);

clearInterval(probe);
// ticks is ~0: the 10ms timer was starved the whole time the loop ran,
// because synchronous CPU work blocks the single thread.
`inline: ~${elapsed}ms, results ${results.length}, timer ticked ${ticks}x while blocked`;
```

Version 2: offloaded to Web Workers (main thread stays free)

Same compute, same probe — but each call runs in its own **Worker** thread. Now the main thread is idle while the workers crunch, so the 10ms timer **keeps ticking**. The three computations also run truly in parallel on separate threads, so total time is closer to one computation, not three.

Deno uses the standard **Web Worker** API. (This cell builds the worker from an in-memory blob, runs it, then cleans up.)

Same heavy loop, but as a worker that messages its result back.

```

const workerSrc = `
  self.onmessage = (e) => {
    let sum = 0;
    for (let i = 0; i < e.data; i++) sum += Math.sqrt(i) * Math.sin(i);
    self.postMessage(sum);
  };
`;
const url = URL.createObjectURL(new Blob([workerSrc], { type: "application/typescript" }));

function runWorker(n: number): Promise<number> {
  return new Promise((resolve) => {
    const w = new Worker(url, { type: "module" });
    w.onmessage = (e) => { resolve(e.data); w.terminate(); };
    w.postMessage(n);
  });
}

// Same probe as the inline version.
let ticks = 0;
const probe = setInterval(() => ticks++, 10);

const t0 = performance.now();
const results = await Promise.all([runWorker(2e7), runWorker(2e7), runWorker(2e7)]);
const elapsed = Math.round(performance.now() - t0);

clearInterval(probe);
URL.revokeObjectURL(url);
// ticks is well above 0 AND elapsed is roughly ONE computation's time,
// not three: the main thread stayed free and the work ran in parallel.
`workers: ~${elapsed}ms, results ${results.length}, timer ticked ${ticks}x while working`;

```

Read the two outputs side by side. Inline: the timer ticked ~0 times (event loop frozen) and time is the sum of all three loops. Workers: the timer kept ticking (main thread free) and time is close to a single loop (parallel). That gap is the entire reason worker threads exist.

Workers have real startup and message-passing cost, so they only pay off for genuinely heavy CPU work. For light tasks the overhead is larger than the savings. For many small tasks, reuse a **pool** of workers instead of spawning one per task.

Production patterns: timeout & retry

Copy-paste building blocks for real code.

```

function delay<T>(ms: number, v?: T) { return new Promise<T>((r) => setTimeout(() => r(v as T), ms)); }

function withTimeout<T>(p: Promise<T>, ms: number): Promise<T> {
  const timeout = new Promise<never>((_, rej) =>
    setTimeout(() => rej(new Error("timeout after " + ms + "ms")), ms));
  return Promise.race([p, timeout]);
}

async function retry<T>(fn: (attempt: number) => Promise<T>, attempts = 3, base = 20):
  Promise<T> {
  let last: unknown;
  for (let i = 0; i < attempts; i++) {
    try { return await fn(i); }
    catch (e) { last = e; if (i < attempts - 1) await delay(base * 2 ** i); }
  }
  throw last;
}

const timedOut = await withTimeout(delay(200, "slow"), 50).catch((e) => e.message);
let calls = 0;
const recovered = await retry((i) => {
  calls++; // count invocations so the output shows the retry actually happened
  return i < 2 ? Promise.reject(new Error("flaky")) : Promise.resolve("ok@" + i);
}, 5);
// calls === 3: failed at i=0 and i=1, succeeded at i=2.
({ timedOut, recovered, calls });

```

Decision cheat-sheet

One future value?	→ Promise / async-await
Many independent I/O ops?	→ Promise.all
Many, but cap the load?	→ bounded pool (mapLimit)
Some allowed to fail?	→ Promise.allSettled
First success wins?	→ Promise.any
Need a timeout?	→ Promise.race
A stream of pushed events?	→ EventEmitter (on/once)
Bridge events→async/await?	→ events.once / events.on
Heavy CPU computation?	→ Worker / worker_threads

That's the full tour. Practice in `exercises/` with `deno task test`.

4 • Streams

A **stream** processes data **incrementally** — chunk by chunk — instead of loading everything into memory at once. Reading a 4 GB log file into a single string would blow up your heap; streaming it lets you handle it a kilobyte at a time at constant memory.

Three core abstractions, all part of the **Web Streams** standard (built into Deno, browsers, and modern Node):

Type	Role	You implement
<code>ReadableStream</code>	source — produces chunks	<code>pull / start</code>
<code>WritableStream</code>	sink — consumes chunks	<code>write</code>
<code>TransformStream</code>	middle — chunk in → chunk out	<code>transform</code>

The other half of the story is **backpressure**: a slow consumer automatically slows a fast producer, so memory stays bounded. We'll see it in action.

We use **Web Streams** throughout (the cross-platform standard). `node:stream` interop is covered at the end.

Creating a ReadableStream

A source is defined by an object with controller callbacks:

- `start(controller)` — runs once; seed initial chunks here.
- `pull(controller)` — called whenever the consumer wants more; enqueue the next chunk. Return a promise to pull asynchronously.
- `controller.enqueue(chunk)` pushes a chunk; `controller.close()` ends the stream.

This stream emits the numbers 1..5, one per `pull`.

```

let next = 1;
const numbers = new ReadableStream<number>({
  pull(controller) {
    if (next > 5) {
      controller.close();
      return;
    }
    controller.enqueue(next++);
  },
});

// Collect the whole stream just to inspect it here.
const collected: number[] = [];
for await (const n of numbers) collected.push(n);
collected; // [1, 2, 3, 4, 5]

```

Two ways to consume

A `ReadableStream` is **async-iterable**, so `for await...of` is the easy path. For finer control use a **reader** (`getReader()`): each `read()` returns `{ value, done }`. A reader **locks** the stream so nothing else can read it concurrently; release it with `releaseLock()`.

```

function makeStream(): ReadableStream<string> {
  const words = ["stream", "data", "incrementally"];
  let i = 0;
  return new ReadableStream({
    pull(controller) {
      if (i >= words.length) controller.close();
      else controller.enqueue(words[i++]);
    },
  });
}

// Manual reader: explicit read loop until done.
const reader = makeStream().getReader();
const seen: string[] = [];
while (true) {
  const { value, done } = await reader.read();
  if (done) break;
  seen.push(value);
}
reader.releaseLock();
seen; // ["stream", "data", "incrementally"]

```

TransformStream — chunk in, chunk out

A `TransformStream` has a **writable** side (you write into it) and a **readable** side (transformed chunks come out). The `transform(chunk, controller)` callback runs per chunk and `enqueue`s zero or more outputs. Here we uppercase each word.

```
const upper = new TransformStream<string, string>({
  transform(chunk, controller) {
    controller.enqueue(chunk.toUpperCase());
  },
});

// Write into the writable side...
const writer = upper.writable.getWriter();
(async () => {
  for (const w of ["alpha", "beta", "gamma"]) await writer.write(w);
  await writer.close();
})();

// ...and read transformed chunks off the readable side.
const out: string[] = [];
for await (const c of upper.readable) out.push(c);
out; // ["ALPHA", "BETA", "GAMMA"]
```

Composing — `.pipeThrough()` and `.pipeTo()`

This is where streams shine. `source.pipeThrough(transform)` returns a new readable (the transform's output), so you **chain** transforms. `.pipeTo(writable)` drains a readable into a sink and resolves when done — and it wires up backpressure for you.


```

function digits(): ReadableStream<number> {
  let n = 1;
  return new ReadableStream({
    pull(c) {
      if (n > 4) c.close();
      else c.enqueue(n++);
    },
  });
}

const double = new TransformStream<number, number>({
  transform: (n, c) => c.enqueue(n * 2),
});
const label = new TransformStream<number, string>({
  transform: (n, c) => c.enqueue("#" + n),
});

// Collect results into a sink via a WritableStream.
const sink: string[] = [];
await digits()
  .pipeThrough(double) // 2, 4, 6, 8
  .pipeThrough(label) // "#2", "#4", "#6", "#8"
  .pipeTo(new WritableStream({ write: (chunk) => void sink.push(chunk) }));

sink; // ["#2", "#4", "#6", "#8"]

```

Backpressure

A stream has an internal **queue** with a *high-water mark*. If the consumer is slow, that queue fills; the producer's `pull` (or a writer's `write`) is then **not** called again until the consumer drains it. The fast producer is throttled to the slow consumer's pace — memory stays bounded automatically.

Below, the producer wants to push as fast as it can, but the sink takes ~30 ms per chunk. We log the timestamp each time `pull` runs: they're spaced out, proving the producer was held back.

```

const gaps: number[] = [];
let last = performance.now();
let value = 0;

const fastSource = new ReadableStream<number>({
  pull(controller) {
    const now = performance.now();
    gaps.push(Math.round(now - last)); // time since previous pull
    last = now;
    if (value >= 4) controller.close();
    else controller.enqueue(value++);
  },
}, { highWaterMark: 1 }); // tiny buffer → backpressure kicks in immediately

// Slow sink: ~30ms to "process" each chunk.
const slowSink = new WritableStream<number>({
  async write() {
    await new Promise((r) => setTimeout(r, 30));
  },
}, { highWaterMark: 1 });

await fastSource.pipeTo(slowSink);
// First pull is ~0ms; later pulls are spaced ~30ms by the slow sink.
gaps;

```

Practical example — line-by-line text processing

The everyday job: take raw bytes, decode to text, split into **lines**, transform each line. We compose three transforms:

1. `TextDecoderStream` — bytes → text (built in).
2. a custom **line splitter** — buffers partial chunks and emits complete lines (note the `flush` to emit the trailing line).
3. a per-line transform — here, number each non-empty line.

This pattern is exactly how you'd stream-parse a large CSV or log file at constant memory.

Source of raw bytes (as if read from a file/socket), split mid-line on purpose.

```

const bytes = new ReadableStream<Uint8Array>({
  start(c) {
    const enc = new TextEncoder();
    c.enqueue(enc.encode("alpha\nbe"));
    c.enqueue(enc.encode("ta\ngamma\n"));
    c.enqueue(enc.encode("delta")); // no trailing newline
    c.close();
  },
});

// Stateful splitter: holds a buffer of the partial last line across chunks.
function splitLines(): TransformStream<string, string> {
  let buffer = "";
  return new TransformStream({
    transform(chunk, controller) {
      buffer += chunk;
      const parts = buffer.split("\n");
      buffer = parts.pop() ?? ""; // last piece may be incomplete
      for (const line of parts) controller.enqueue(line);
    },
    flush(controller) {
      if (buffer) controller.enqueue(buffer); // emit trailing line
    },
  });
}

let lineNo = 0;
const numberLines = new TransformStream<string, string>({
  transform: (line, c) => c.enqueue(`${++lineNo}: ${line}`),
});

const lines: string[] = [];
await bytes
  .pipeThrough(new TextDecoderStream())
  .pipeThrough(splitLines())
  .pipeThrough(numberLines)
  .pipeTo(new WritableStream({ write: (l) => void lines.push(l) }));

lines; // ["1: alpha", "2: beta", "3: gamma", "4: delta"]

```

node:stream interop

Older Node APIs (`fs.createReadStream` , HTTP request/response bodies) use Node's **object-mode / classic** streams, not Web Streams. Bridge between the two worlds with the static helpers on `node:stream`'s `Readable` :

- `Readable.toWeb(nodeStream)` → a `ReadableStream` you can `pipeThrough` .
- `Readable.fromWeb(webStream)` → a Node `Readable` for legacy APIs.

(`Writable.toWeb` / `Duplex.toWeb` exist too.) Prefer Web Streams for new code; reach for these only at the boundary with old Node libraries.

```
import { Readable } from "node:stream";

// A classic Node Readable (push-based, object mode here).
const nodeStream = Readable.from(["one", "two", "three"]);

// Convert it to a Web ReadableStream and consume it the standard way.
const web = Readable.toWeb(nodeStream) as ReadableStream<string>;
const collected: string[] = [];
for await (const chunk of web) collected.push(chunk);

collected; // ["one", "two", "three"]
```

Takeaways

- Stream when data is **large or unbounded** — constant memory beats buffering everything.
- `ReadableStream` (source) / `WritableStream` (sink) / `TransformStream` (middle) are the three building blocks.
- Consume with `for await...of` (easy) or `getReader()` (manual control; remember `releaseLock`).
- Compose with `.pipeThrough(transform)` and finish with `.pipeTo(sink)`.
- **Backpressure** is automatic — a slow sink throttles a fast source so memory stays bounded.
- Bridge to legacy Node with `Readable.toWeb` / `Readable.fromWeb`, but prefer Web Streams.

That's the tour. Practice in `exercises/` with `deno task test`.